```
public:
    typedef Key key_type;
    typedef Value mapped_type;
    typedef Hash hash_type;

    threadsafe_lookup_table(
        unsigned num_buckets=19,Hash const& hasher_=Hash()):
        buckets(num_buckets),hasher(hasher_)
    {
        for(unsigned i=0;i<num_buckets;++i)
        {
            buckets[i].reset(new bucket_type);
        }
    }

    threadsafe_lookup_table(threadsafe_lookup_table const& other)=delete;
    threadsafe_lookup_table& operator=(
        threadsafe_lookup_table const& other)=delete;

    Value value_for(Key const& key,
                    Value const& default_value=Value()) const
    {
        return get_bucket(key).value_for(key,default_value);     <-8
    }

    void add_or_update_mapping(Key const& key,Value const& value)
    {
        get_bucket(key).add_or_update_mapping(key,value);     <-9
    }

    void remove_mapping(Key const& key)
    {
        get_bucket(key).remove_mapping(key);     <-10
    }
};
```

This implementation uses a `std::vector<std::unique_ptr<bucket_type>>` ❻ to hold the buckets, which allows the number of buckets to be specified in the constructor. The default is 19, which is an arbitrary prime number; hash tables work best with a prime number of buckets. Each bucket is protected with an instance of `boost::shared_mutex` ❶ to allow many concurrent reads or a single call to either of the modification functions *per bucket*.

Because the number of buckets is fixed, the `get_bucket()` function ❼ can be called without any locking ❽, ❾, ❿, and then the bucket mutex can